

EE446 Lab 8

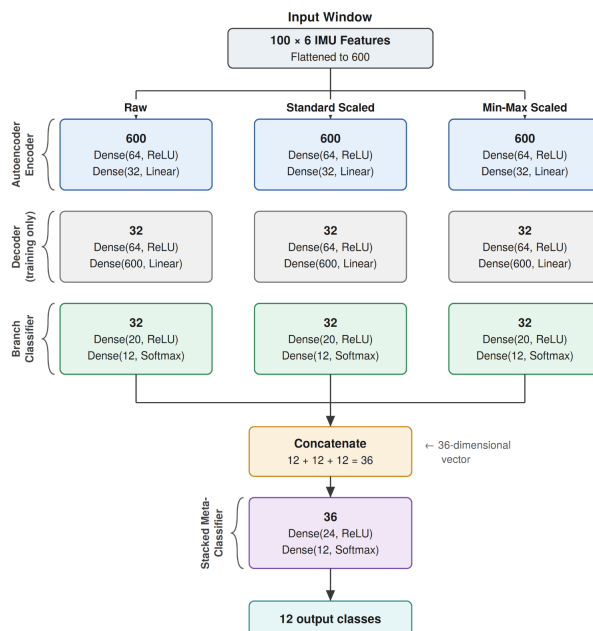
Phina Penteriani

Edge impulse link: <https://studio.edgeimpulse.com/public/1013715/live>

Screenshot:

```
Sampling...
Timing: DSP 159 ms, inference 553 us, anomaly 0 ms, postprocessing 66 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 163 ms, inference 552 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 163 ms, inference 552 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 163 ms, inference 552 us, anomaly 0 ms, postprocessing 68 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 163 ms, inference 552 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 160 ms, inference 574 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
Starting inferencing in 2 seconds...
Sampling...
```

Part 2:



2. Pruning is applied with the TensorFlow Model Optimization Toolkit (TF-MOT) function `prune_low_magnitude()`, which removes the least important weights by forcing many parameters to zero. It uses a polynomial sparsity schedule that ramps sparsity from 10% to 80% over 500 steps, letting the model adjust gradually rather than all at once.

The pruning wrappers are then removed. They barely reduce model size on their own, and quantization libraries expect unmodified Keras models, so the wrappers—which mainly hold pruning metadata and mask logic—are stripped before QAT.

QAT is performed with `quantize_model()`, which inserts simulated quantization so the model trains with quantization noise. The weights are therefore optimized to stay accurate once actually quantized.

Because the model is pruned, the pruning masks are reapplied after QAT: training can otherwise push zeroed parameters back to nonzero values, so reapplying the masks keeps them at zero.

Finally, the model is converted to an int8 TFLite model. A representative data generator estimates the input data range, letting the converter pick accurate scaling factors.

On a resource-constrained device, far less memory and compute are available than on the training machine. Pruning and int8 quantization shrink the model substantially while preserving most of the baseline's accuracy—keeping the most important parameters and compressing them.

```
Final prediction: Lying down
Class index: 2
Confidence score: 0.5078
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.3281, 0.3281, 0.0000, 0.0000, 0.0000, 0.3281, 0.0000, 0.0000, 0.0000, 0.0117, 0.007
Standard-scaled model scores: 0.9901, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0
Min-max-scaled model scores: 0.0000, 0.0000, 0.4100, 0.0000, 0.0977, 0.0090, 0.2031, 0.1562, 0.0000, 0.0000, 0
Meta-classifier scores: 0.3437, 0.0234, 0.5078, 0.0117, 0.0000, 0.0156, 0.0000, 0.0078, 0.0000, 0.0117, 0.0703,
-----
Final prediction: Lying down
Class index: 2
Confidence score: 0.5078
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.3281, 0.3281, 0.0000, 0.0000, 0.0000, 0.3281, 0.0000, 0.0000, 0.0000, 0.0117, 0.007
Standard-scaled model scores: 0.9901, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0
Min-max-scaled model scores: 0.0000, 0.0000, 0.4100, 0.0000, 0.0977, 0.0090, 0.2031, 0.1562, 0.0000, 0.0000, 0
Meta-classifier scores: 0.3437, 0.0234, 0.5078, 0.0117, 0.0000, 0.0156, 0.0000, 0.0078, 0.0000, 0.0117, 0.0703,
-----
Final prediction: Lying down
Class index: 2
Confidence score: 0.5078
-----
```

3. 

This output is acceptable for the resting board. It sits completely still and the prediction is "Lying down." Once I started moving it, however, only two other classes were ever predicted: "Standing up" and "Knees bending." Nothing else came up.

"Standing up" appeared whenever I moved the board along the horizontal axis, while "Knees bending" showed up when I moved it along the vertical axis. This mostly makes sense to me, though I had expected "Standing up" to register when the board was held still above its starting position rather than while it was actively in motion.

To me this suggests that clearer usage instructions, especially around how the board should be oriented and moved, would make the deployment more reliable. It also shows that understanding your dataset matters just as much as building the model itself.

Question 4: Deployment Behavior on the Arduino

The deployment produced some unexpected predictions. The board landed on "Standing up" most of the time, but only while it was in constant motion, and just 3 of the 12 classes were predicted at all. I believe this comes down to how differently the board was being moved compared to the sensor that gathered the training data. I was swinging the board on a cord, whereas that data was most likely collected from a sensor worn by a person.

A few other factors could be contributing as well, such as differences in board orientation and the noise profile of our sensor versus the original one. Differences like these may push the model toward favoring certain classes, and it is also possible that the motions I tested simply never activated the feature patterns tied to the remaining classes.